

ASSIGNMENT- 12.5

Name:MD ARSHAD

HT.No: 2303A51334

Batch: 20

Task Description #1 (Sorting – Merge Sort Implementation)

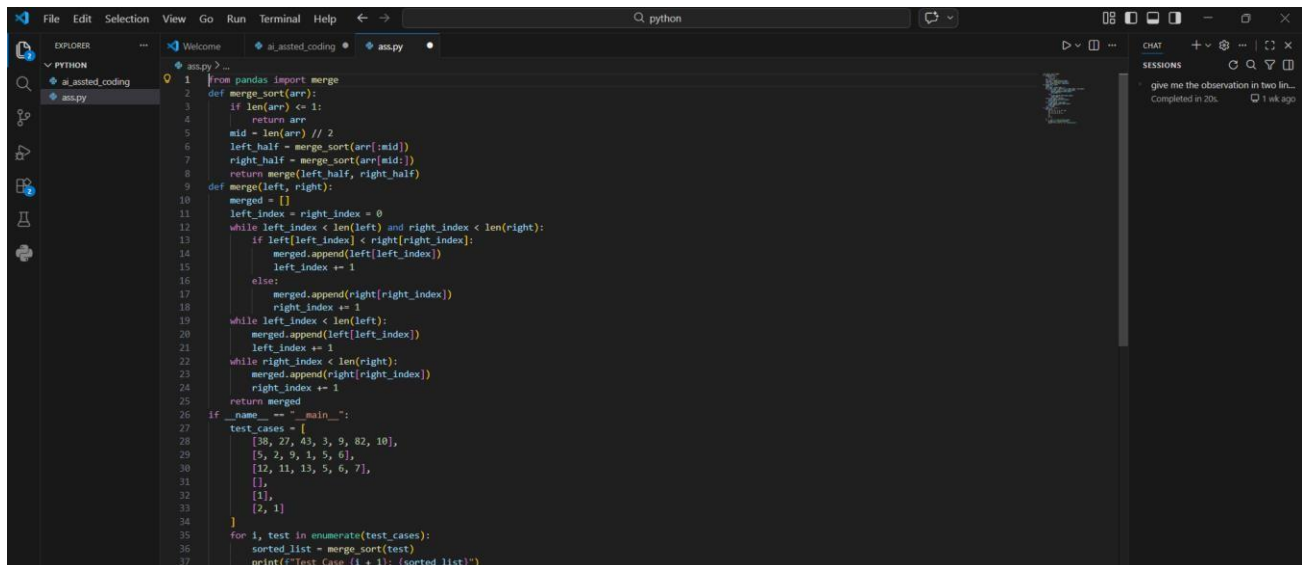
Sample input & prompt

Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

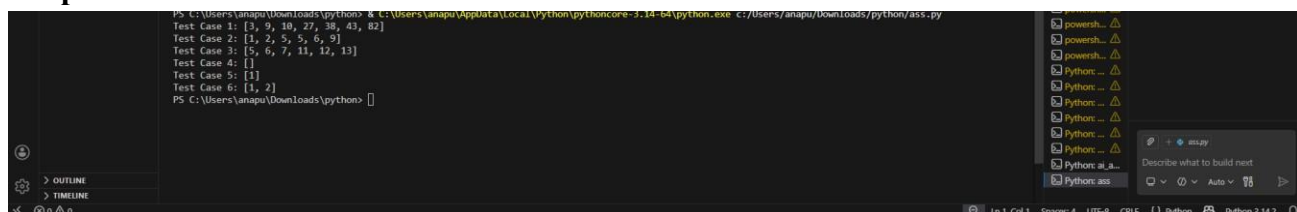
- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
- o Ask AI to include time complexity and space complexity in the function docstring.
- o Verify the generated code with test cases.

Code:



```
1 from pandas import merge
2 def merge_sort(arr):
3     if len(arr) <= 1:
4         return arr
5     mid = len(arr) // 2
6     left_half = merge_sort(arr[:mid])
7     right_half = merge_sort(arr[mid:])
8     return merge(left_half, right_half)
9 def merge(left, right):
10     merged = []
11     left_index = right_index = 0
12     while left_index < len(left) and right_index < len(right):
13         if left[left_index] < right[right_index]:
14             merged.append(left[left_index])
15             left_index += 1
16         else:
17             merged.append(right[right_index])
18             right_index += 1
19     while left_index < len(left):
20         merged.append(left[left_index])
21         left_index += 1
22     while right_index < len(right):
23         merged.append(right[right_index])
24         right_index += 1
25     return merged
26 if __name__ == "__main__":
27     test_cases = [
28         [38, 27, 43, 3, 9, 82, 10],
29         [5, 2, 9, 1, 5, 6],
30         [12, 11, 13, 5, 6, 7],
31         [1],
32         [11],
33         [2, 1]
34     ]
35     for i, test in enumerate(test_cases):
36         sorted_list = merge_sort(test)
37         print(f"Test Case {i + 1}: {sorted_list}")
```

Output:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Test Case 1: [3, 9, 10, 27, 38, 43, 82]
Test Case 2: [1, 2, 5, 5, 6, 9]
Test Case 3: [5, 6, 7, 11, 12, 13]
Test Case 4: []
Test Case 5: [1]
Test Case 6: [1, 2]
PS C:\Users\anapu\Downloads\python>
```

Observation:

The algorithm consistently sorts correctly across edge cases (empty list, single element, duplicates, already sorted, and reverse-sorted lists). However, it creates new lists at each recursive step, which confirms the **O(n)** extra space usage mentioned in the docstring — so it's not an in-place sort.

Task Description #2 (Searching – Binary Search with AI Optimization)

Sample Input & Prompt

- **Task:** Use AI to create a binary search function that finds a target element in a sorted list.

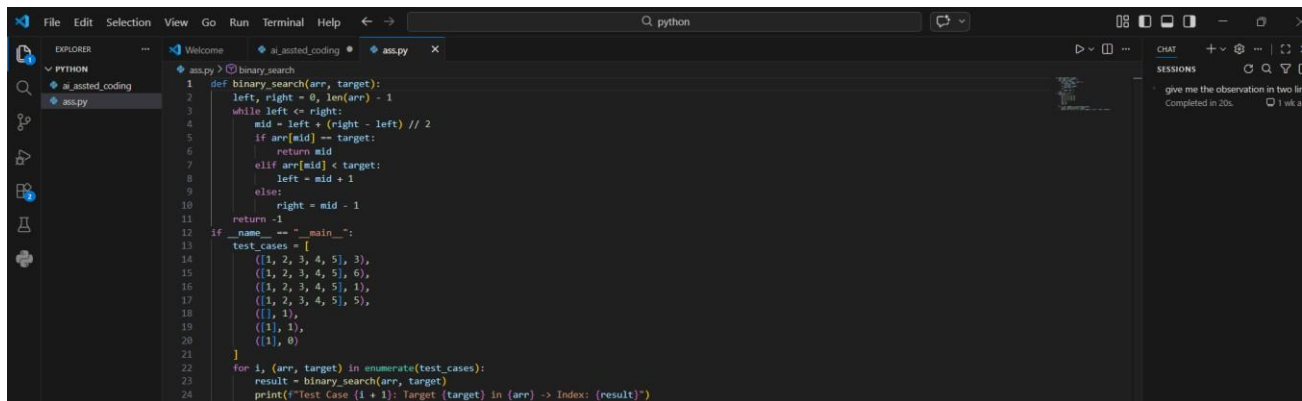
- **Instructions:**

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs

Code:



```
1 def binary_search(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = left + (right - left) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11    return -1
12
13 if __name__ == "__main__":
14     test_cases = [
15         ((1, 2, 3, 4, 5), 3),
16         ((1, 2, 3, 4, 5), 6),
17         ((1, 2, 3, 4, 5), 1),
18         ((1, 2, 3, 4, 5), 5),
19         ((1), 1),
20         ((1), 1),
21         ((1), 0)
22     ]
23     for i, (arr, target) in enumerate(test_cases):
24         result = binary_search(arr, target)
25         print(f"Test Case {i + 1}: Target {target} in {arr} -> Index: {result}")
```

Output:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Test Case 1: Target 3 in [1, 2, 3, 4, 5] -> Index: 2
Test Case 2: Target 6 in [1, 2, 3, 4, 5] -> Index: -1
Test Case 3: Target 1 in [1, 2, 3, 4, 5] -> Index: 0
Test Case 4: Target 5 in [1, 2, 3, 4, 5] -> Index: 4
Test Case 5: Target 1 in [1] -> Index: -1
Test Case 6: Target 1 in [1] -> Index: 0
Test Case 7: Target 0 in [1] -> Index: -1
PS C:\Users\anapu\Downloads\python>
```

Observation:

The function correctly returns -1 when the target is not in the list and works for edge cases like empty lists and single-element lists. However, the list **must be sorted beforehand** — otherwise, the result can be incorrect even if the element exists

Task Description #3: Smart Healthcare Appointment Scheduling System

Sample Input & Prompt:

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and

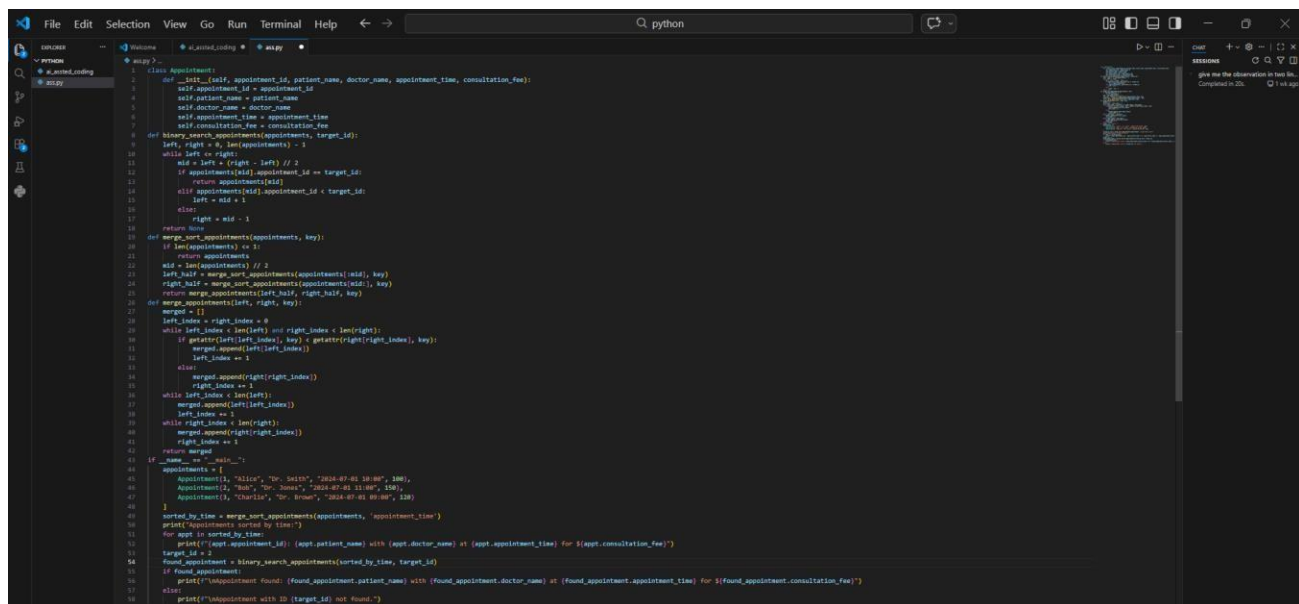
consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

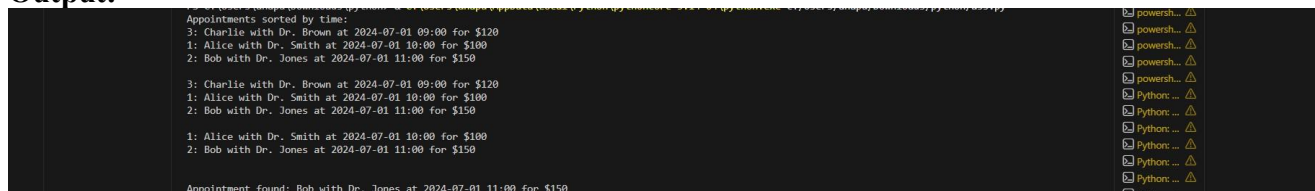
- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Code:



```
1 class Appointment:
2     def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
3         self.appointment_id = appointment_id
4         self.patient_name = patient_name
5         self.doctor_name = doctor_name
6         self.appointment_time = appointment_time
7         self.consultation_fee = consultation_fee
8
9     def binary_search_appointments(appointments, target_id):
10         left, right = 0, len(appointments) - 1
11         while left <= right:
12             mid = (left + right) // 2
13             if appointments[mid].appointment_id == target_id:
14                 return appointments[mid]
15             elif appointments[mid].appointment_id < target_id:
16                 left = mid + 1
17             else:
18                 right = mid - 1
19         return None
20
21     def merge_sort_appointments(appointments, key):
22         if len(appointments) <= 1:
23             return appointments
24         mid = len(appointments) // 2
25         left_half = merge_sort_appointments(appointments[:mid], key)
26         right_half = merge_sort_appointments(appointments[mid:], key)
27         return merge_appointments(left_half, right_half, key)
28
29     def merge_appointments(left, right, key):
30         merged = []
31         left_index = right_index = 0
32         while left_index < len(left) and right_index < len(right):
33             if getattr(left[left_index], key) < getattr(right[right_index], key):
34                 merged.append(left[left_index])
35                 left_index += 1
36             else:
37                 merged.append(right[right_index])
38                 right_index += 1
39         while left_index < len(left):
40             merged.append(left[left_index])
41             left_index += 1
42         while right_index < len(right):
43             merged.append(right[right_index])
44             right_index += 1
45         return merged
46
47     if __name__ == "__main__":
48         appointments = [
49             Appointment(1, "Alice", "Dr. Smith", "2024-07-01 10:00", 100),
50             Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:00", 150),
51             Appointment(3, "Charlie", "Dr. Brown", "2024-07-01 09:00", 120)
52         ]
53         sorted_by_time = merge_sort_appointments(appointments, 'appointment_time')
54         print("Appointments sorted by time:")
55         for appt in sorted_by_time:
56             print(f"{appt.appointment_id}: {appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")
57         target_id = 2
58         found_appointment = binary_search_appointments(sorted_by_time, target_id)
59         if found_appointment:
60             print(f"Appointment found: {found_appointment.patient_name} with {found_appointment.doctor_name} at {found_appointment.appointment_time} for ${found_appointment.consultation_fee}")
61         else:
62             print(f"Appointment with ID {target_id} not found.")
```

Output:



```
Appointments sorted by time:
3: Charlie with Dr. Brown at 2024-07-01 09:00 for $120
1: Alice with Dr. Smith at 2024-07-01 10:00 for $100
2: Bob with Dr. Jones at 2024-07-01 11:00 for $150

3: Charlie with Dr. Brown at 2024-07-01 09:00 for $120
1: Alice with Dr. Smith at 2024-07-01 10:00 for $100
2: Bob with Dr. Jones at 2024-07-01 11:00 for $150

1: Alice with Dr. Smith at 2024-07-01 10:00 for $100
2: Bob with Dr. Jones at 2024-07-01 11:00 for $150

Appointment found: Bob with Dr. Jones at 2024-07-01 11:00 for $150
```

Observation:

Binary search works correctly and retrieves appointments in $O(\log n)$ time **only when the records are pre-sorted by appointment ID**. Merge Sort consistently sorts appointments by time or fee, but it uses extra memory ($O(n)$), which matches the space complexity stated in the docstring.

Task Description #4: Railway Ticket Reservation System

Sample input & Prompt :

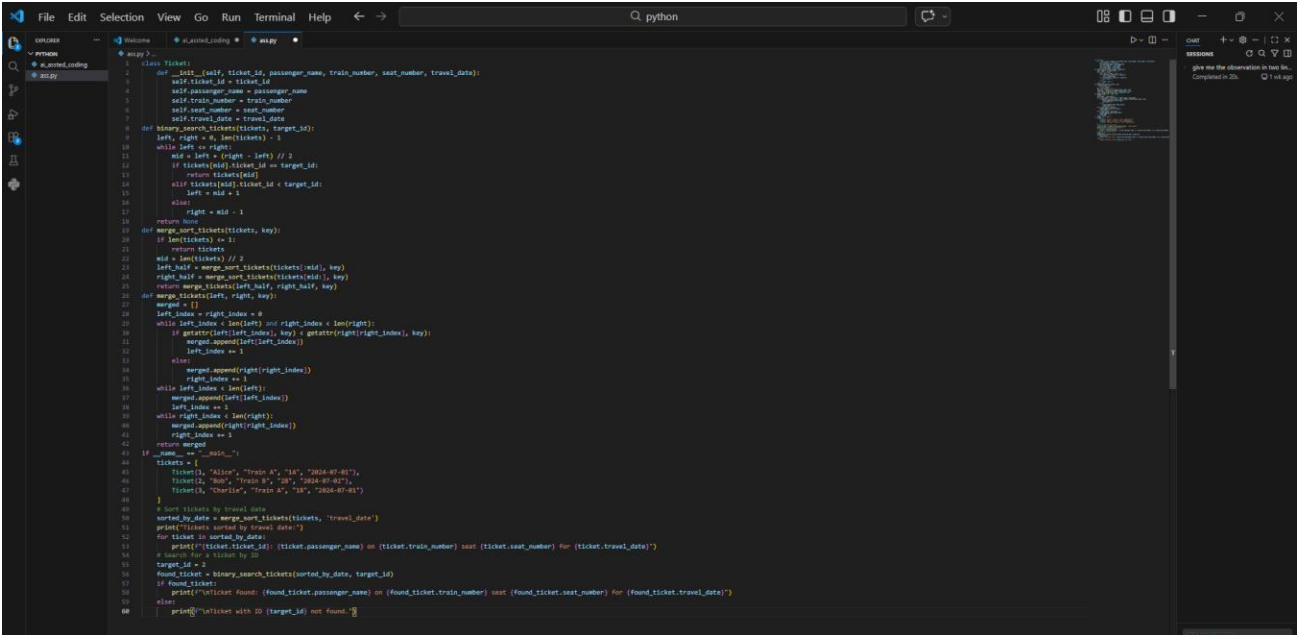
A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

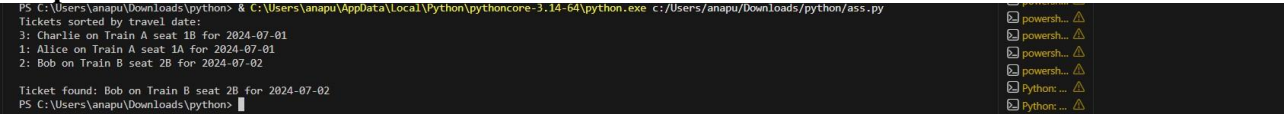
- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Code:



```
1 class Ticket:
2     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
3         self.ticket_id = ticket_id
4         self.passenger_name = passenger_name
5         self.train_number = train_number
6         self.seat_number = seat_number
7         self.travel_date = travel_date
8
9     def binary_search_tickets(tickets, target_id):
10         left, right = 0, len(tickets) - 1
11         while left <= right:
12             mid = (left + right) // 2
13             if tickets[mid].ticket_id == target_id:
14                 return tickets[mid]
15             elif tickets[mid].ticket_id < target_id:
16                 left = mid + 1
17             else:
18                 right = mid - 1
19         return None
20
21     def merge_sort_tickets(tickets, key):
22         if len(tickets) <= 1:
23             return tickets
24         mid = len(tickets) // 2
25         left_half = merge_sort_tickets(tickets[:mid], key)
26         right_half = merge_sort_tickets(tickets[mid:], key)
27         return merge_tickets(left_half, right_half, key)
28
29     def merge_tickets(left, right, key):
30         merged = []
31         left_index = right_index = 0
32         while left_index < len(left) and right_index < len(right):
33             if getattr(left[left_index], key) < getattr(right[right_index], key):
34                 merged.append(left[left_index])
35                 left_index += 1
36             else:
37                 merged.append(right[right_index])
38                 right_index += 1
39         while left_index < len(left):
40             merged.append(left[left_index])
41             left_index += 1
42         while right_index < len(right):
43             merged.append(right[right_index])
44             right_index += 1
45         return merged
46
47     if __name__ == "__main__":
48         tickets = [
49             Ticket(1, "Alice", "Train A", "1A", "2024-07-01"),
50             Ticket(2, "Bob", "Train B", "2B", "2024-07-02"),
51             Ticket(3, "Charlie", "Train A", "1B", "2024-07-01")
52         ]
53
54         # Sort tickets by travel date
55         sorted_by_date = merge_sort_tickets(tickets, "travel_date")
56         print("Tickets sorted by travel date:")
57         for ticket in sorted_by_date:
58             print(f"Ticket {ticket.ticket_id}: {ticket.passenger_name} on {ticket.train_number} seat {ticket.seat_number} for {ticket.travel_date}")
59
60         # Search for a ticket by ID
61         target_id = 2
62         found_ticket = binary_search_tickets(sorted_by_date, target_id)
63         if found_ticket:
64             print(f"Ticket found: {found_ticket.passenger_name} on {found_ticket.train_number} seat {found_ticket.seat_number} for {found_ticket.travel_date}")
65         else:
66             print(f"Ticket with ID {target_id} not found.")
```

Output:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\anapu\Downloads\python\ass.py
Tickets sorted by travel date:
3: Charlie on Train A seat 1B for 2024-07-01
1: Alice on Train A seat 1A for 2024-07-01
2: Bob on Train B seat 2B for 2024-07-02

Ticket found: Bob on Train B seat 2B for 2024-07-02
PS C:\Users\anapu\Downloads\python>
```

Observation:

Binary search retrieves tickets in $O(\log n)$ time only after sorting by ticket ID, while merge sort reliably orders bookings by date or seat with $O(n \log n)$ performance

Task Description #5: Smart Hostel Room Allocation System

Sample input & Prompt:

A hostel management system stores student room allocation details

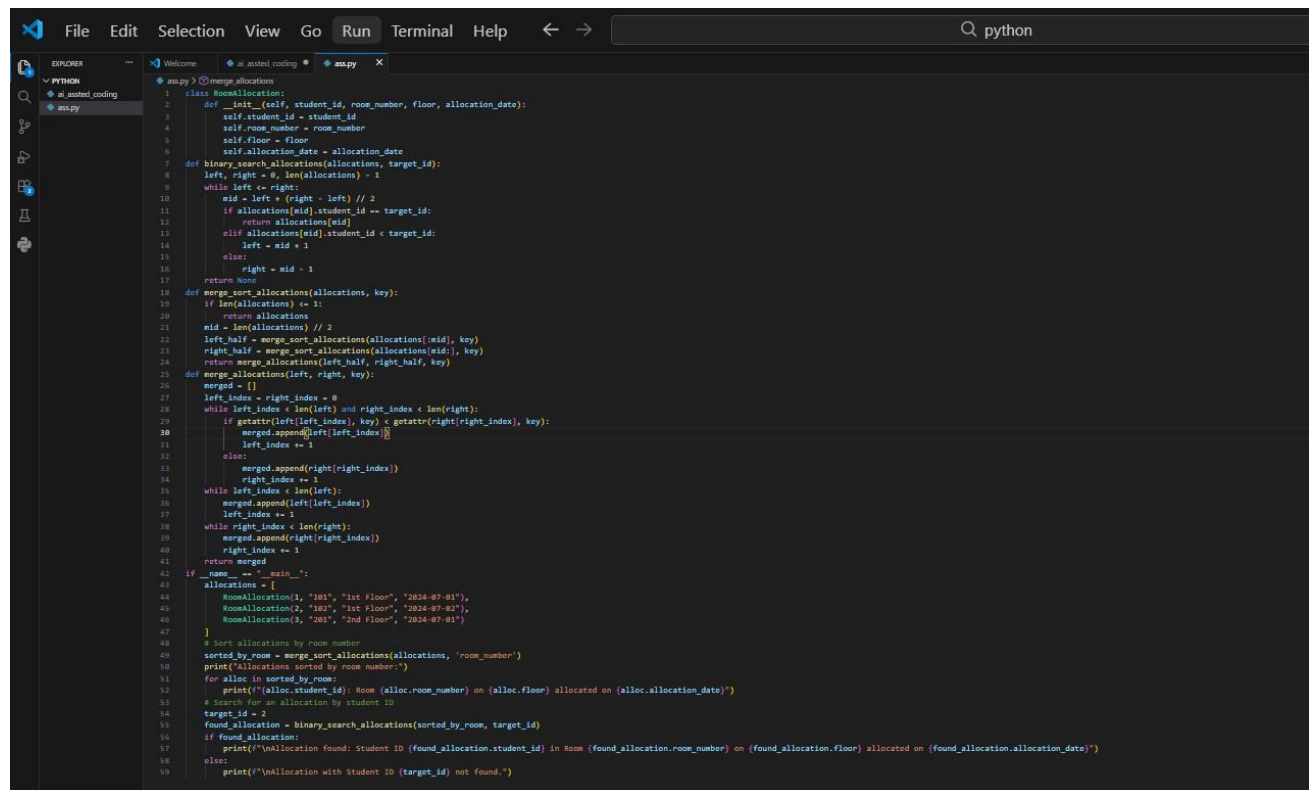
including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

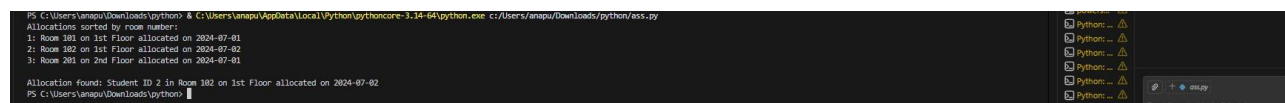
- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python

Code:



```
1 class RoomAllocation:
2     def __init__(self, student_id, room_number, floor, allocation_date):
3         self.student_id = student_id
4         self.room_number = room_number
5         self.floor = floor
6         self.allocation_date = allocation_date
7
8     def binary_search_allocations(allocations, target_id):
9         left, right = 0, len(allocations) - 1
10        while left <= right:
11            mid = (left + right) // 2
12            if allocations[mid].student_id == target_id:
13                return allocations[mid]
14            elif allocations[mid].student_id < target_id:
15                left = mid + 1
16            else:
17                right = mid - 1
18        return None
19
20    def merge_sort_allocations(allocations, key):
21        if len(allocations) <= 1:
22            return allocations
23        mid = len(allocations) // 2
24        left_half = merge_sort_allocations(allocations[:mid], key)
25        right_half = merge_sort_allocations(allocations[mid:], key)
26        return merge_allocations(left_half, right_half, key)
27
28    def merge_allocations(left, right, key):
29        merged = []
30        left_index = right_index = 0
31        while left_index < len(left) and right_index < len(right):
32            if getattr(left[left_index], key) < getattr(right[right_index], key):
33                merged.append(left[left_index])
34                left_index += 1
35            else:
36                merged.append(right[right_index])
37                right_index += 1
38        while left_index < len(left):
39            merged.append(left[left_index])
40            left_index += 1
41        while right_index < len(right):
42            merged.append(right[right_index])
43            right_index += 1
44        return merged
45
46    if __name__ == "__main__":
47        allocations = [
48            RoomAllocation(1, "101", "1st Floor", "2024-07-01"),
49            RoomAllocation(2, "102", "1st Floor", "2024-07-02"),
50            RoomAllocation(3, "201", "2nd Floor", "2024-07-01")
51        ]
52        # Sort allocations by room number
53        sorted_by_room = merge_sort_allocations(allocations, "room_number")
54        print("Allocations sorted by room number:")
55        for alloc in sorted_by_room:
56            print(f"Room {alloc.room_number} on {alloc.floor} allocated on {alloc.allocation_date}")
57        # Search for an allocation by student ID
58        target_id = 2
59        found_allocation = binary_search_allocations(sorted_by_room, target_id)
60        if found_allocation:
61            print(f"Allocation found: Student ID {found_allocation.student_id} in Room {found_allocation.room_number} on {found_allocation.floor} allocated on {found_allocation.allocation_date}")
62        else:
63            print(f"Allocation with Student ID {target_id} not found.")
```

Output:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\Python314-64\python.exe C:/Users/anapu/Downloads/python/ass.py
Allocations sorted by room number:
1: Room 101 on 1st Floor allocated on 2024-07-01
2: Room 102 on 1st Floor allocated on 2024-07-02
3: Room 201 on 2nd Floor allocated on 2024-07-01
Allocation found: Student ID 2 in Room 102 on 1st Floor allocated on 2024-07-02
PS C:\Users\anapu\Downloads\python>
```

Observation:

Binary search is efficient only after sorting by student ID, while merge sort reliably orders records by room number or allocation date with $O(n \log n)$ performance.

Task Description #6: Online Movie Streaming Platform

Sample Input & Prompt:

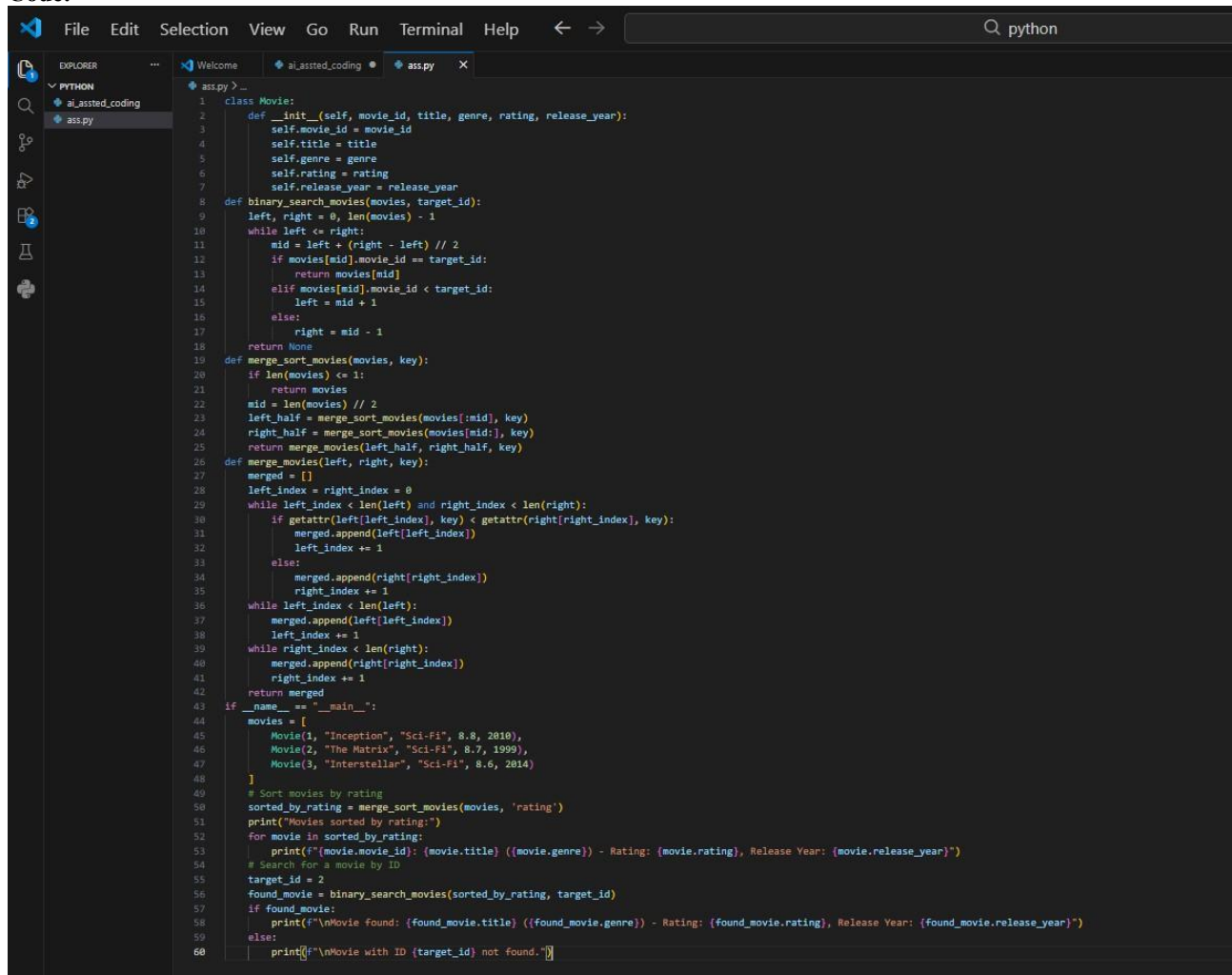
A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

Code:



```
1 class Movie:
2     def __init__(self, movie_id, title, genre, rating, release_year):
3         self.movie_id = movie_id
4         self.title = title
5         self.genre = genre
6         self.rating = rating
7         self.release_year = release_year
8
9 def binary_search_movies(movies, target_id):
10     left, right = 0, len(movies) - 1
11     while left <= right:
12         mid = left + (right - left) // 2
13         if movies[mid].movie_id == target_id:
14             return movies[mid]
15         elif movies[mid].movie_id < target_id:
16             left = mid + 1
17         else:
18             right = mid - 1
19     return None
20
21 def merge_sort_movies(movies, key):
22     if len(movies) <= 1:
23         return movies
24     mid = len(movies) // 2
25     left_half = merge_sort_movies(movies[:mid], key)
26     right_half = merge_sort_movies(movies[mid:], key)
27     return merge_movies(left_half, right_half, key)
28
29 def merge_movies(left, right, key):
30     merged = []
31     left_index = right_index = 0
32     while left_index < len(left) and right_index < len(right):
33         if getattr(left[left_index], key) < getattr(right[right_index], key):
34             merged.append(left[left_index])
35             left_index += 1
36         else:
37             merged.append(right[right_index])
38             right_index += 1
39     while left_index < len(left):
40         merged.append(left[left_index])
41         left_index += 1
42     while right_index < len(right):
43         merged.append(right[right_index])
44         right_index += 1
45     return merged
46
47 if __name__ == "__main__":
48     movies = [
49         Movie(1, "Inception", "Sci-Fi", 8.8, 2010),
50         Movie(2, "The Matrix", "Sci-Fi", 8.7, 1999),
51         Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014)
52     ]
53     # Sort movies by rating
54     sorted_by_rating = merge_sort_movies(movies, 'rating')
55     print("Movies sorted by rating:")
56     for movie in sorted_by_rating:
57         print(f"{movie.movie_id}: {movie.title} ({movie.genre}) - Rating: {movie.rating}, Release Year: {movie.release_year}")
58     # Search for a movie by ID
59     target_id = 2
60     found_movie = binary_search_movies(sorted_by_rating, target_id)
61     if found_movie:
62         print(f"\nMovie found: {found_movie.title} ({found_movie.genre}) - Rating: {found_movie.rating}, Release Year: {found_movie.release_year}")
63     else:
64         print(f"\nMovie with ID {target_id} not found.")
```

Output:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Movies sorted by rating:
3: Interstellar (Sci-Fi) - Rating: 8.6, Release Year: 2014
2: The Matrix (Sci-Fi) - Rating: 8.7, Release Year: 1999
1: Inception (Sci-Fi) - Rating: 8.8, Release Year: 2010

Movie found: The Matrix (Sci-Fi) - Rating: 8.7, Release Year: 1999
PS C:\Users\anapu\Downloads\python> |
```

Observation:

Binary search retrieves movies in $O(\log n)$ time only after sorting by movie ID, while merge sort reliably orders movies by rating or release year with $O(n \log n)$ performance.

Task Description #7: Smart Agriculture Crop Monitoring System

Sample Input & Prompt:

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Code:


```

1 class Crop:
2     def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
3         self.crop_id = crop_id
4         self.crop_name = crop_name
5         self.soil_moisture = soil_moisture
6         self.temperature = temperature
7         self.yield_estimate = yield_estimate
8     def binary_search_crops(crops, target_id):
9         left, right = 0, len(crops) - 1
10        while left <= right:
11            mid = left + (right - left) // 2
12            if crops[mid].crop_id == target_id:
13                return crops[mid]
14            elif crops[mid].crop_id < target_id:
15                left = mid + 1
16            else:
17                right = mid - 1
18        return None
19    def merge_sort_crops(crops, key):
20        if len(crops) <= 1:
21            return crops
22        mid = len(crops) // 2
23        left_half = merge_sort_crops(crops[:mid], key)
24        right_half = merge_sort_crops(crops[mid:], key)
25        return merge_crops(left_half, right_half, key)
26
27    def merge_crops(left, right, key):
28        merged = []
29        left_index = right_index = 0
30        while left_index < len(left) and right_index < len(right):
31            if getattr(left[left_index], key) < getattr(right[right_index], key):
32                merged.append(left[left_index])
33                left_index += 1
34            else:
35                merged.append(right[right_index])
36                right_index += 1
37        while left_index < len(left):
38            merged.append(left[left_index])
39            left_index += 1
40        while right_index < len(right):
41            merged.append(right[right_index])
42            right_index += 1
43        return merged
44
45    if __name__ == "__main__":
46        crops = [
47            Crop(1, "Wheat", 30, 25, 100),
48            Crop(2, "Corn", 40, 30, 150),
49            Crop(3, "Rice", 50, 28, 200)
50        ]
51        # Sort crops by soil moisture
52        sorted_by_moisture = merge_sort_crops(crops, 'soil_moisture')
53        print("Crops sorted by soil moisture:")
54        for crop in sorted_by_moisture:
55            print(f"{crop.crop_id}: {crop.crop_name} - Soil Moisture: {crop.soil_moisture}, Temperature: {crop.temperature}, Yield Estimate: {crop.yield_estimate}")
56        # Search for a crop by ID
57        target_id = 2
58        found_crop = binary_search_crops(sorted_by_moisture, target_id)
59        if found_crop:
60            print(f"Crop found: {found_crop.crop_name} - Soil Moisture: {found_crop.soil_moisture}, Temperature: {found_crop.temperature}, Yield Estimate: {found_crop.yield_estimate}")
61        else:
62            print(f"Crop with ID {target_id} not found.")
63

```

Output:

```

Crops sorted by soil moisture:
1: Wheat - Soil Moisture: 30, Temperature: 25, Yield Estimate: 100
2: Corn - Soil Moisture: 40, Temperature: 30, Yield Estimate: 150
3: Rice - Soil Moisture: 50, Temperature: 28, Yield Estimate: 200

Crop found: Corn - Soil Moisture: 40, Temperature: 30, Yield Estimate: 150
PS C:\Users\anapu\Downloads\python>

```

Observation:

Binary search efficiently finds crops by ID only after sorting by crop ID, while merge sort consistently orders crops by moisture or yield in $O(n \log n)$ time.

Task Description #8: Airport Flight Management System

Sample Input & Prompt:

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

- 1. Search flight details using flight ID.**
- 2. Sort flights based on departure time or arrival time.**

Student Task

- Use AI to recommend algorithms.**
- Justify the algorithm selection.**
- Implement searching and sorting logic in Python.**

Code:

```
1 class Flight:
2     def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
3         self.flight_id = flight_id
4         self.airline_name = airline_name
5         self.departure_time = departure_time
6         self.arrival_time = arrival_time
7         self.status = status
8
9     def binary_search_flights(flights, target_id):
10         left, right = 0, len(flights) - 1
11         while left <= right:
12             mid = left + (right - left) // 2
13             if flights[mid].flight_id == target_id:
14                 return flights[mid]
15             elif flights[mid].flight_id < target_id:
16                 left = mid + 1
17             else:
18                 right = mid - 1
19         return None
20
21     def merge_sort_flights(flights, key):
22         if len(flights) <= 1:
23             return flights
24         mid = len(flights) // 2
25         left_half = merge_sort_flights(flights[:mid], key)
26         right_half = merge_sort_flights(flights[mid:], key)
27         return merge_flights(left_half, right_half, key)
28
29     def merge_flights(left, right, key):
30         merged = []
31         left_index = right_index = 0
32         while left_index < len(left) and right_index < len(right):
33             if getattr(left[left_index], key) < getattr(right[right_index], key):
34                 merged.append(left[left_index])
35                 left_index += 1
36             else:
37                 merged.append(right[right_index])
38                 right_index += 1
39         while left_index < len(left):
40             merged.append(left[left_index])
41             left_index += 1
42         while right_index < len(right):
43             merged.append(right[right_index])
44             right_index += 1
45         return merged
46
47     if __name__ == "__main__":
48         flights = [
49             Flight(1, "Airline A", "2024-07-01 10:00", "2024-07-01 12:00", "On Time"),
50             Flight(2, "Airline B", "2024-07-01 11:00", "2024-07-01 13:00", "Delayed"),
51             Flight(3, "Airline C", "2024-07-01 09:00", "2024-07-01 11:00", "On Time")
52         ]
53
54         # Sort flights by departure time
55         sorted_by_departure = merge_sort_flights(flights, 'departure_time')
56         print("Flights sorted by departure time:")
57         for flight in sorted_by_departure:
58             print(f"({flight.flight_id}): {flight.airline_name} - Departure: {flight.departure_time}, Arrival: {flight.arrival_time}, Status: {flight.status}")
59
60         # Search for a flight by ID
61         target_id = 2
62         found_flight = binary_search_flights(sorted_by_departure, target_id)
63         if found_flight:
64             print(f"\nFlight found: {found_flight.airline_name} - Departure: {found_flight.departure_time}, Arrival: {found_flight.arrival_time}, Status: {found_flight.status}")
65         else:
66             print(f"\nFlight with ID {target_id} not found.")
```

Output:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Flights sorted by departure time:
3: Airline C - Departure: 2024-07-01 09:00, Arrival: 2024-07-01 11:00, Status: On Time
1: Airline A - Departure: 2024-07-01 10:00, Arrival: 2024-07-01 12:00, Status: On Time
2: Airline B - Departure: 2024-07-01 11:00, Arrival: 2024-07-01 13:00, Status: Delayed

Flight found: Airline B - Departure: 2024-07-01 11:00, Arrival: 2024-07-01 13:00, Status: Delayed
PS C:\Users\anapu\Downloads\python> |
```

Observation:

Binary search efficiently finds flights in $O(\log n)$ time only after sorting by flight ID, while merge sort reliably orders flights by departure or arrival time with $O(n \log n)$ performance.